

Projeto IA 25/26

Autores

Kirill Egorkin
Tomás Carvalhosa

Primeiro Capítulo: Estrutura do Tabuleiro

Importância da Performance

A estrutura do tabuleiro é crucial para a performance, especialmente com o algoritmo **MCTS**. Este método exige um volume elevado de simulações para cada jogada, tornando a eficiência de dados um requisito fundamental.

Implementação Bitboard

Utilizamos **Bitboards** (inteiros de 64 bits) para representar o estado do jogo.

- Operações *Drop* e *Pop* são efetuadas via *bit-shifting* aritmético.
- **Redução de 95% no tempo de execução.**
- De 20s para apenas 0,9s.

Segundo Capítulo: Pesquisa MCTS

Passos da Execução

O algoritmo Monte Carlo Tree Search percorre um ciclo iterativo fundamental:

- Seleção
- Expansão
- Rollout
- Backpropagation

Variantes e Parâmetros

Para além da implementação básica, o MCTS pode ser otimizado através de:

- Mudança das **Regras de Rollout**
- Ajuste de **Regras de Expansão**
- Alteração do valor de **C** no UCT para manipular a Exploração

Variantes de MCTS

Exploração via C

C baixo: Afunila a exploração em jogadas com alta taxa de vitória.

C elevado: Prioriza a exploração de nós menos visitados.

Expansão & Rollout

Expansão Aleatória: Escolhe k jogadas aleatórias em vez de todas.

Heuristic Rollout: Simulação guiada por regras específicas em vez de aleatória.

Outras Otimizações

Epsilon Greedy: Rollout inteligente com probabilidade de movimentos aleatórios.

Depth Limited: Interrompe a simulação ao atingir um limite de profundidade.

Variantes obtidas ao aplicar mudanças ao algoritmo-base.

Resultados torneio



Gerar os datasets

Processo de Geração

Para gerar os datasets a serem treinados criamos um script que coloca a **versão base do MCTS** a jogar contra si próprio (durante **750 jogos**) que, a cada passo memoriza o tabuleiro e a ação que escolheu registrando os mesmos num ficheiro CSV.

Dataset do Torneio

Visão Geral do Dataset

Para gerar o dataset do torneio entre os **sete agentes do MCTS** a ideia geral é a mesma, sendo a grande diferença o facto de cada par jogar **2 vezes** entre si, resultando num dataset bastante rico em informação aprendendo várias estratégias de jogo.

Dataset do Torneio Perturbado

Processo de Geração e Recuperação

Este processo pega no dataset do torneio anterior e realiza os seguintes passos:

- Lê linha a linha o ficheiro CSV e reconstrói o tabuleiro.
- Faz entre 2 a 6 jogadas aleatórias e descarta estados terminais.
- Três melhores agentes MCTS definem a jogada de recuperação após o baralhamento.
- Apaga as colunas desnecessárias para não confundir o ID3

Algoritmo ID3: Íris

Contexto e Processamento

O dataset íris serviu como teste para a **lógica matemática do ID3** para ser possível continuar com o treino do Popout.

Como este dataset tem dados contínuos, utilizou-se a ferramenta **qcut** da biblioteca **Pandas** para organizar esses dados em secções, tornando-os compatíveis com o ID3.

Algoritmo ID3: Popout

Implementação e Testes

Para o treino dos datasets do **Popout** usamos a implementação **pura do ID3** programando a **Entropia** e o **Ganho de informação**.

Algoritmo ID3: Popout Torneio

Ajustes no Treino do Dataset

- **Exclusão de colunas auxiliares** para simplificar a tomada de decisão do ID3.
- **Cálculo da entropia inicial** pelas jogadas e iteração por casa do tabuleiro (maior Ganho de Informação = Raiz).
- **Divisão em 3 sub-árvores** baseadas no estado do atributo: "Vazio", "X" ou "O".
- **Processo recursivo** para cada sub-árvore, calculando o Ganho de Informação na descida.

Algoritmo ID3: Popout Torneio Perturbado

Treino do Dataset Perturbado

- **Seleção do Agente Alvo:** Determinamos o agente alvo para fundamentar as escolhas da árvore.
- **Divisão de Dados:** Separação entre dados de treino e validação (essenciais para a poda).
- **Hiperparâmetros:** Definição de profundidade máxima e ganho mínimo de informação.
- **Validação Pós-Poda:** O script testa a árvore modificada contra o conjunto de validação.

Resultados Torneio ID3



Interface

A interface é uma das partes mais importantes do jogo.

No arranque o script carrega o motor do jogo, a IA de pesquisa em tempo real MCTS e a árvore de decisão treinada.

Game loop:

- Desenha o estado atual do tabuleiro no terminal
- Chama as funções de verificação de vencedor ou se é possível declarar um empate
- Se for esse o caso, pede o input ao jogador humano
- Aplica a jogada com uma função própria para esse efeito

Interface Cont.

Tratamento do input:

A função **get_human_move()** divide a string (ex: drop / 3), verifica a validade do número e cruza com a função **legal_moves()**.

MCTS vs ID3:

- **MCTS:** Instanciado a cada jogada, analisando o estado ao vivo e simulando em tempo real.
- **ID3:** Converte o tabuleiro para dicionário e processa via função **classificar()**.
- **Fallback:** Se a árvore encontrar uma situação não treinada, executa o primeiro movimento legal.

Conclusão

Com este projeto conseguimos concluir que uma boa exploração dos algoritmos de implementação e treino de uma árvore de decisão é imperativa para a descoberta do melhor agente possível.

Com esta diversidade de algoritmos e a exploração dos mesmos ficámos com um conhecimento mais aprofundado dos métodos necessários para criar uma IA capaz de combater um oponente quer seja ele artificial ou humano.